Farmers Market Database: 200 SQL Interview Questions (up to, but not including, JOINs)

Topics: SELECT & derived columns · WHERE / AND / OR / NOT · IN / NOT IN · BETWEEN · LIKE · CONCAT / CONCAT_WS · INSTR / LOCATE · LEFT / RIGHT / SUBSTRING / SUBSTRING_INDEX · TRIM / LENGTH / CHAR_LENGTH / REPLACE / REVERSE · NULL handling (IS NULL, COALESCE, IFNULL, NULLIF) · CASE and SUM(CASE) · numeric and date functions · aggregates and DISTINCT · GROUP BY / HAVING · ORDER BY / LIMIT / OFFSET · order of execution · subqueries · statistics · interview logic

SQL flavour: MySQL 8.0 (MySQL Workbench). The answers use only core SQL (no CTEs), so almost all of them also run on the MySQL 5.6 editor on InterviewBit. Where 5.6 behaves differently, the answer says so.

Rules

- **No JOINs.** Each question needs only one table. When a question uses two tables, connect them with a **subquery** (`IN` , `EXISTS` , scalar or correlated).
- Give every derived column a clear alias.
- For **"What will this return?"** questions, work out the answer on paper first, then run the query to check it.
- The answers are at the end of the file.

Setup: in Workbench, run `farmers_market_setup.sql` . It creates the `farmers_market` database and loads all 9 tables from the CSVs in `csv/` .

> **About the data:** it was copied from the InterviewBit output, which cuts long results short. `customer_purchases` stops at 2019-09-04 for product 2, and `market_date_info` stops at 2020-07-25. If you run these queries on InterviewBit's full data, your numbers will differ from the ones quoted in the answers.

Schema Reference and Quirks

Table	Columns	Quirks worth knowing
`market_date_info`	market_date, market_day, market_week, market_year, market_start_time, market_end_time, special_notes, market_season, market_min_temp, market_max_temp, market_rain_flag, market_snow_flag	`market_week`, `market_year` and both temperature columns are **VARCHAR**. Start and end times are text such as `'8:00 AM'`. Temperatures are NULL on most days. `special_notes` is always NULL.
`booth`	booth_number, booth_price_level (A/B/C), booth_description, booth_type	Several descriptions contain a **double space** before the `-`. Booths 8 and 9 have **identical** descriptions.
`vendor`	vendor_id, vendor_name, vendor_type, vendor_owner_first_name, vendor_owner_last_name	'Hernández' has an accented **á**. Apostrophes are missing ('Annies Pies', 'Chriss ...').
`customer`	customer_id, customer_first_name, customer_last_name, customer_zip	`customer_zip` is VARCHAR. Two customers have the last name Diaz.
`product_category`	product_category_id, product_category_name	Names contain `&` and `( )`.
`product`	product_id, product_name, product_size, product_category_id, product_qty_type	Size is NULL for product 14 and a single **space** for product 15. Qty type is NULL for products 14 and 15. Some sizes are bare numbers ('10', '6'). There's a typo: 'Resuable'.
`vendor_booth_assignments`	vendor_id, booth_number, market_date	In summer 2019, vendor 4 **and** vendor 7 are both assigned to booth 11 on the same dates.
`vendor_inventory`	market_date, quantity, vendor_id, product_id, original_price	Only vendors 7 and 8 have inventory.
`customer_purchases`	product_id, vendor_id, market_date, customer_id, quantity, cost_to_customer_per_qty, transaction_time	`transaction_time` is TIME. On 2019-07-13, two customers paid **3.00** for product 2 instead of the usual 3.49.

Wednesday markets run 4:00–7:00 PM and Saturday markets run 8:00 AM–2:00 PM. Prices are in $, weights in lbs and temperatures in °F.

Section 1: SELECT, Derived Columns & Aliases

Q1. Finance wants a line-level sales report. From `customer_purchases`, show `market_date`, `customer_id`, `product_id`, `quantity`, `cost_to_customer_per_qty` and a column `line_total` (quantity × price) rounded to 2 decimals.

Q2. Assume a 5.3% sales tax. For every row in `customer_purchases`, show `line_total`, `tax_amount` and `amount_payable`, each rounded to 2 decimals. Then explain why adding the rounded `line_total` and rounded `tax_amount` can differ from the rounded `amount_payable` by a cent.

Q3. The manager wants to know how much stock (by value) vendors bring. From `vendor_inventory`, show `market_date`, `vendor_id`, `product_id`, `quantity`, `original_price` and `stock_value`. Return only the 10 highest-value rows.

Q4. From `market_date_info`, show `market_date`, the min and max temperature, `temp_swing` (max − min) and `avg_temp_c` (the average of min and max, converted to Celsius with `(F − 32) × 5/9` and rounded to 1 decimal). Both temperature columns are VARCHAR, so does the arithmetic still work? What shows up on days with no reading?

Q5. Products 1 and 2 are sold by the pound. For each purchase in `customer_purchases`, show `qty_kg` (1 lb = 0.4536 kg, rounded to 3 decimals) and `price_per_kg` (rounded to 2 decimals).

Q6. What will this return?

```
SELECT 7/2 AS a, 7 DIV 2 AS b, 7 % 2 AS c, -7 DIV 2 AS d, -7 % 2 AS e, 1 + '1 lb' AS f;
```

Q7. What happens when you run this query? Fix it.

```
SELECT quantity * cost_to_customer_per_qty AS line_total
FROM customer_purchases
WHERE line_total > 30;
```

Q8. Loyalty points equal the whole dollars spent on a line. Cents are dropped, not rounded, so a $39.91 line earns 39 points. From `customer_purchases`, show `line_total` and `reward_points`.

Q9. The market is planning a 12.5% markup, with the new price rounded **up** to the next nickel ($0.05). For example, 6.99 × 1.125 = 7.86375 becomes 7.90. From `vendor_inventory`, show the distinct `product_id`, `original_price` and `new_price`.

Q10. List every vendor with two constant columns: `market_name` = 'Harrisonburg Farmers Market' and `onboarding_year` = 2019, plus the vendor name.

Q11. What will this return?

```
SELECT product_id, product_id + 0.5, product_id / 0, product_name + 0
FROM product
WHERE product_id = 7;
```

Q12. A printed receipt needs headers with spaces in them. From `customer_purchases`, show **"Customer ID"**, **"Market Day"** (the weekday name of `market_date`) and **"Amount ($)"** (the rounded line total). Sort by "Amount ($)" from highest to lowest.

Section 2: WHERE, AND / OR / NOT

Q13. Find all Wednesday markets in 2019 that had rain. `market_year` is VARCHAR. Does `market_year = 2019` still work?

Q14. The manager asked for "Saturday markets with rain or snow" and got this query:

```
SELECT * FROM market_date_info
WHERE market_day = 'Saturday' AND market_rain_flag = 1 OR market_snow_flag = 1;
```

On this data the output happens to look right. Why is the query still wrong? Fix it.

Q15. Find the Jalapeno pepper purchases (product 2) that were sold at a price other than the regular 3.49.

Q16. Find the transactions made in the last 30 minutes before closing. Wednesday markets close at 7:00 PM and Saturday markets at 2:00 PM. Use only `customer_purchases`.

Q17. Products 1 and 2 are weighed, so a whole-number quantity such as exactly 2.00 lbs is unusual. Find the purchases whose quantity has no fractional part, without using LIKE.

Q18. Find the market days that were neither rainy nor snowy. Write it twice: once with `NOT (… OR …)` and once using De Morgan's law.

Q19. What does this return, and why isn't it 126 (128 rows minus the 2 days with a min temp of 50)?

```
SELECT COUNT(*) FROM market_date_info WHERE market_min_temp <> 50;
```

Q20. Find the customers in zip 22801 whose last name starts with a letter from A to M. Use comparison operators, not LIKE. Why does `customer_last_name <= 'M'` miss 'Messina'?

Q21. Find purchase lines worth more than $20 that were made before noon.

Q22. Find the market days where **exactly one** of rain or snow happened. Write it with `XOR`, then without it.

Q23. What does this return? Does it run, and is there any hidden cost?

```
SELECT * FROM customer WHERE customer_zip = 22801;
```

Q24. A colleague wanted the markets that start before noon and wrote this:

```
SELECT * FROM market_date_info WHERE market_start_time < '12:00 PM';
```

It returns 0 rows. Explain why, then fix it.

Q25. Find the products whose `product_qty_type` is **not** 'unit'. The products where the qty type is unknown should also appear. Why does `product_qty_type <> 'unit'` alone miss some rows?

Q26. The goal is product 2 purchases on 2019-07-13 by customer 2 or customer 10. Find the bug and fix it:

```
SELECT * FROM customer_purchases
WHERE market_date = '2019-07-13' AND customer_id = 2 OR customer_id = 10 AND product_id = 2;
```

Section 3: IN, NOT IN, BETWEEN

Q27. Find the purchases made by customers 1, 4 and 14 at Saturday markets.

Q28. Using `IN`, list the vendors whose type is 'Prepared Foods' or 'Eggs & Meats'.

Q29. List the booths that have **never** been assigned to a vendor. Use `booth` with a subquery on `vendor_booth_assignments`.

Q30. What will this return? Why? Fix it.

```
SELECT product_name FROM product WHERE product_qty_type NOT IN ('unit', NULL);
```

Q31. Using BETWEEN on `market_date`, find all purchases from July 2019. Then answer: if `market_date` were a DATETIME, why would `BETWEEN '2019-07-01' AND '2019-07-31'` lose most of July 31st?

Q32. Find the markets whose min temperature was between 40 and 55 °F. The column is VARCHAR. Is `BETWEEN '40' AND '55'` safe? When would it break?

Q33. What will this return?

```
SELECT COUNT(*) FROM customer_purchases WHERE quantity BETWEEN 5 AND 3;
```

Q34. Find the transactions made between 11:00 AM and 1:00 PM (inclusive) at Saturday markets.

Q35. Find the customers whose first name falls alphabetically from 'B' through 'D'. What happens to 'Deanna', 'Dawn' and 'Darrell' with `BETWEEN 'B' AND 'D'`? Fix the query.

Q36. Without a JOIN, find the products in the categories 'Fresh Fruits & Vegetables' or 'Eggs & Meat (Fresh or Frozen)'.

Q37. Find the vendors that have **never** brought inventory to the market.

Q38. Using `NOT BETWEEN`, find the purchases that were **not** made during the 2019 selling window ('2019-06-01' to '2019-10-30'). How many rows do you expect, and why?

Section 4: LIKE

Q39. Find all organic products. Does `LIKE '%organic%'` also match 'Organic' with a capital O? Why?

Q40. Find the products sold in a jar or a bag, meaning their names end with 'Jar' or 'Bag'.

Q41. Find the vendors whose name contains '&'. Then find the vendor types that contain a colon.

Q42. Using LIKE only, find the customers whose first name has exactly 3 letters.

Q43. Find the customers whose last name has 'a' as the **second** letter.

Q44. How would you search `product_name` for a literal underscore `_` or percent sign `%`? Write the query for underscore, and show two ways to escape it.

Q45. Find the vendors whose business name **begins with** the owner's first name (e.g. 'Marcos Peppers' for Marco).

Q46. Find the vendors whose business name contains the owner's **last name** anywhere.

Q47. Data-quality check: find the booths whose description contains a double space.

Q48. Marketing asks for the 'Red Potatoes' products. Compare `product_name = 'Red Potatoes'` with `product_name LIKE 'Red Potatoes%'`. How many rows does each return, and which fits the request?

Q49. Find the products whose size **starts with a digit** (e.g. '8 oz', '1.5 lbs', '10'). Write it without REGEXP, then with REGEXP.

Q50. Find the customers whose first and last names start with the same letter.

Section 5: CONCAT & Formatting

Q51. Build a display name such as **'Connor, Jane'** for every customer, sorted by last name, then first name.

Q52. Generate loyalty-card emails in the form first initial + last name + customer_id + '@fmarket.com', all lowercase. Customer 1 gets `jconnor1@fmarket.com`.

Q53. Build a product label such as **'Habanero Peppers - Organic (medium, lbs)'**. What happens to products 14 and 15? Rewrite it so that no label comes out NULL.

Q54. What will this return?

```
SELECT CONCAT('Booth-', 7), CONCAT('A', NULL, 'B'), CONCAT_WS('-', 'A', NULL, 'B'),
CONCAT_WS(NULL, 'A', 'B');
```

Q55. Build booth codes such as **'A-01'**, **'C-09'** and **'B-12'** from `booth_price_level` and a zero-padded `booth_number`.

Q56. Create a market-day label such as **'Sat, 02-Mar-2019 (Wk 09)'** from `market_date_info`.

Q57. Build an invoice line string for each purchase in this form: **'Cust 14 bought 0.99 lbs @ $6.99 = $6.92'**.

Q58. Mask customer last names for a public leaderboard. 'Connor' becomes **'C*'**: keep the first letter and replace the rest with stars.

Q59. Build a social-media handle from `vendor_name`: all lowercase, ' & ' replaced by '*and*' and the remaining spaces replaced by '_'. 'Chriss Sustainable Eggs & Meats' becomes

`chriss_sustainable_eggs_and_meats`.

Q60. What does `SELECT 'Eggs' || 'Meat';` return in MySQL, and why? How do you concatenate correctly?

Section 6: INSTR, LOCATE, LEFT, RIGHT, SUBSTRING, SUBSTRING_INDEX

Q61. Split `product_name` into `base_name` and `variant` at ' - '. For example, 'Habanero Peppers - Organic' becomes 'Habanero Peppers' and 'Organic'. When a product has no dash, `base_name` should be the full name and `variant` should be NULL. Use INSTR with LEFT and SUBSTRING, **not** SUBSTRING_INDEX.

Q62. Solve Q61 with SUBSTRING_INDEX. Why does `SUBSTRING_INDEX(product_name, ' - ', -1)` return a wrong variant for 'Eggs' or 'Sweet Corn'?

Q63. Extract the **first word** of every product name. Make sure one-word names such as 'Eggs' still work, then explain what `LEFT(name, INSTR(name, ' ') - 1)` returns for 'Eggs'.

Q64. Extract the **last word** of every product name in two ways: with SUBSTRING_INDEX, and with REVERSE + INSTR.

Q65. From `booth_description`, extract the table-size text after the dash (e.g. 'Small square table size'). Return NULL when there is no dash, and make sure the double spaces don't break the result.

Q66. Extract the first word of every booth description ('First', 'Second', 'Third', 'Back'), then count the booths for each word.

Q67. From `market_start_time` (e.g. '8:00 AM' or '4:00 PM'), extract the hour as a number and the AM/PM part **without hard-coding positions**. Remember that '12:00 PM' is longer than '8:00 AM'.

Q68. Without STR_TO_DATE, convert `market_start_time` and `market_end_time` to 24-hour numbers using LEFT/INSTR/RIGHT and CASE. Then compute the market's length in hours. Which date has an unusual length?

Q69. For sizes such as '8 oz', '1.5 lbs', '1 lb' and '1 dozen', split `product_size` into `size_value` (the text before the first space) and `size_unit` (the text after it). Only take rows whose size contains a space and starts with a digit.

Q70. What will this return?

```
SELECT LEFT('Apple Pie', 0), RIGHT('Apple Pie', 3), SUBSTRING('Apple Pie', 0, 3),
       SUBSTRING('Apple Pie', -3), SUBSTRING('Apple Pie', 7), MID('Apple Pie', 2, 3);
```

Q71. What will this return?

```
SELECT INSTR('Apple Pie', 'p'), LOCATE('p', 'Apple Pie'), LOCATE('p', 'Apple Pie', 4),
       INSTR('Apple Pie', 'z'), POSITION('Pie' IN 'Apple Pie');
```

Q72. For product names with at least two spaces, find the position of the **second** space.

Q73. For product names with **exactly three words**, extract the middle word. 'Cut Zinnias Bouquet' gives 'Zinnias'.

Q74. From `vendor_type`, extract the text before ':' when there is one ('Fresh Variety'). Otherwise return the full type.

Q75. Extract the hour and minute from `transaction_time` in two ways: with LEFT/SUBSTRING, and with HOUR()/MINUTE(). Why does the string version work on a TIME column?

Q76. Get the year and the month number from `market_date` using LEFT/SUBSTRING, then using YEAR()/MONTH(). Which approach is safer, and why?

Q77. Data check: find the products whose name contains ' - ' but whose part after the dash is not 'Organic', 'Jar' or 'Bag'.

Q78. Build a customer code such as **'JC-801-01'**: the initials, then the **last 3** digits of the zip, then the customer_id padded to 2 digits.

Section 7: TRIM, LENGTH, CHAR_LENGTH, REPLACE, REVERSE, UPPER/LOWER

Q79. What will this return?

```
SELECT LENGTH('Hernández'), CHAR_LENGTH('Hernández'), LENGTH(' '), CHAR_LENGTH(''),
LENGTH(NULL);
```

Q80. Using the difference between LENGTH and CHAR_LENGTH, find the vendors whose name or owner name contains non-ASCII (accented) characters.

Q81. Clean `booth_description` by collapsing double spaces into single spaces. Show the original length, the cleaned length and the cleaned text, only for the rows that changed.

Q82. Count the words in each product name. Count ' - ' as a separator, not as a word, so 'Habanero Peppers - Organic' has 3 words.

Q83. Count how many times the letter 'e' (upper or lower case) appears in each customer's full name. Why does `REPLACE(name, 'e', '')` alone undercount?

Q84. Find the customer first names that are palindromes, ignoring case.

Q85. What will this return?

```
SELECT TRIM('  Eggs  '), TRIM(LEADING '0' FROM '00120'), TRIM(TRAILING 's' FROM 'Peppers'),
       TRIM(BOTH '-' FROM '--Jar--'), LTRIM('  a '), RTRIM(' a  ');
```

Q86. Standardise `product_size` to sentence case: first letter uppercase and the rest lowercase ('medium' becomes 'Medium').

Q87. Find the product whose size is **blank but not NULL**. Why does `product_size = ''` find it on some servers but not others?

Q88. The search box is case-insensitive. Find the vendors with 'PIES' in their name. Then make the search **case-sensitive** and explain the new result.

Q89. Find the product(s) with the longest name, including ties.

Q90. Find the position of the **last** 'e' in each vendor name using REVERSE, and return 0 when there is none.

Section 8: NULL Handling

Q91. Find the market dates with no recorded temperature. Why does `WHERE market_min_temp = NULL` return nothing?

Q92. Show `market_date`, `market_min_temp` and `market_max_temp`, and display 'Not recorded' where a temperature is NULL.

Q93. What will this return?

```
SELECT NULL = NULL, NULL <=> NULL, NULL IS NULL, 1 + NULL, COALESCE(NULL, NULL, 'x'),
       IFNULL(NULL, 0), NULLIF(5, 5), NULLIF(5, 6), GREATEST(3, NULL);
```

Q94. Compare `COUNT(*)`, `COUNT(market_min_temp)` and `COUNT(special_notes)` in `market_date_info`, and explain each number.

Q95. Compute the average min temperature in three ways: `AVG(market_min_temp)`, `SUM(market_min_temp)/COUNT(*)` and `AVG(COALESCE(market_min_temp, 0))`. Which one is right, and why are the other two wrong?

Q96. Build a size label that shows 'Size not specified' when `product_size` is NULL **or** blank.

Q97. What does `SELECT 10/0;` return in MySQL, and how is that different from SQL Server or PostgreSQL? Rewrite "revenue per customer on each market date" so it can never divide by zero.

Q98. Find the products where size **or** qty type is missing, counting NULL, empty and space-only values as missing.

Q99. What will this return? The table has 23 products and 7 of them are 'lbs'.

```
SELECT COUNT(*) FROM product WHERE product_qty_type <> 'lbs';
```

Q100. Sort the markets by min temperature from lowest to highest. Where does MySQL put the NULLs? Rewrite the query so that NULLs come **last** and the temperatures sort as numbers.

Q101. Show 'Rain' or 'Dry' for each market day using `IF()`. How are `IFNULL` and `COALESCE` different?

Q102. What will this return?

```
SELECT COUNT(*), SUM(market_min_temp), AVG(market_min_temp), MAX(market_min_temp)
FROM market_date_info WHERE market_year = '2020';
```

Section 9: CASE and SUM(CASE)

Q103. Classify each market day by its max temperature: 'Cold' (< 50), 'Mild' (50–69), 'Warm' (≥ 70) and 'Unknown' (NULL).

Q104. Why does this CASE never return 'Warm'? Fix it.

```
CASE WHEN market_max_temp >= 50 THEN 'Mild'
     WHEN market_max_temp >= 70 THEN 'Warm'
     ELSE 'Cold' END
```

Q105. Label each purchase line as 'Small' (< $5), 'Medium' ($5–$20) or 'Large' (> $20). Show the line count and revenue for each label.

Q106. Pivot the table so that each product is one row, with the columns `sales_2019` and `sales_2020` for revenue.

Q107. For each customer, show `wed_lines` and `sat_lines` (the number of purchase lines on Wednesday and on Saturday).

Q108. What percentage of purchase lines were made before 12:00 noon? Write it with SUM(CASE), then with AVG of a boolean.

Q109. What will this return?

```
SELECT CASE WHEN NULL = NULL THEN 'equal' ELSE 'not equal' END,
       CASE NULL WHEN NULL THEN 'match' ELSE 'no match' END,
       CASE WHEN 1 = 2 THEN 'x' END;
```

Q110. Show the distinct market seasons in the business order Spring, then Summer/Early Fall, then Late Fall/Holiday. Do it with CASE in ORDER BY, then with FIELD().

Q111. Create a `weather` column with the values 'Rain & Snow', 'Rain', 'Snow' or 'Clear'.

Q112. For each season, find the percentage of market days that had rain.

Q113. Count the booths for each price level as **one row** with the columns `level_a`, `level_b` and `level_c`.

Q114. Flag each product 2 purchase as 'Discounted' or 'Full price' by comparing the price paid with the **highest** price ever paid for product 2. Use a scalar subquery inside CASE.

Q115. Use a single WHERE clause with CASE to return the Saturday purchases over $20 and the Wednesday purchases over $10.

Q116. Build a data-quality status for each product. The checks come in this order: 'Missing qty type' (NULL), 'Missing size' (NULL or blank), 'Size is a bare number' (the size contains only digits) and 'OK'. Which products get which status, and why does the order of the WHEN clauses matter?

Section 10: Numeric & Date Functions

Q117. What will this return?

```
SELECT ROUND(6.985, 2), ROUND(2.5), ROUND(-2.5), ROUND(1234.5, -2),
       TRUNCATE(6.999, 2), CEIL(4.01), FLOOR(-4.01), ROUND(2.5e0);
```

Q118. A cash-only stall rounds every bill to the nearest **quarter** ($0.25). Show `line_total` and `cash_total`.

Q119. For each market date, show the weekday, the month name, the quarter and the ISO week. Is the stored `market_week` the same as `WEEK(market_date)`? Which WEEK mode matches it?

Q120. As of 2020-10-10, how many days has it been since each customer's last purchase?

Q121. Find the purchases made in the first 7 days of any month.

Q122. Convert `market_start_time` and `market_end_time` to real TIME values with STR_TO_DATE, compute the market's length with TIMEDIFF, and sort the markets by their true start time.

Q123. For each transaction, compute the minutes since the market opened (16:00 on Wednesdays, 08:00 on Saturdays).

Q124. Using `DAYOFWEEK()`, find the purchases made on a Saturday. What number is Saturday, and which day is 1?

Q125. Show each purchase quantity as whole pounds plus leftover ounces (1 lb = 16 oz, rounded). For example, 2.18 lb is 2 lb 3 oz.

Q126. What will this return?

```
SELECT MOD(10, 3), 10 MOD 3, MOD(-10, 3), POWER(2, 10), SQRT(16), ABS(-3.5), SIGN(-3.5);
```

Section 11: Aggregates & DISTINCT

Q127. Build a one-row KPI summary of `customer_purchases`: purchase lines, distinct customers, total revenue, average line value, and the smallest and largest line value.

Q128. A **visit** is one (customer, market_date) pair. How many visits are there? Count them without a subquery.

Q129. What will this return?

```
SELECT COUNT(product_size), COUNT(DISTINCT product_size), COUNT(*) FROM product;
```

Q130. What was the true average price per pound paid for Jalapenos (product 2)? Compare `AVG(cost_to_customer_per_qty)` with the weighted average `SUM(q × p) / SUM(q)`, and say which one is correct.

Q131. Why does this query fail in MySQL 8? Fix it.

```
SELECT DISTINCT customer_id FROM customer_purchases ORDER BY market_date;
```

Q132. For vendor 7, how many distinct market days did it bring stock (`vendor_inventory`), and how many days did it record at least one sale (`customer_purchases`)?

Q133. For each vendor, show the total stock value brought across all dates and the total quantity brought.

Q134. What will this return, and why?

```
SELECT MAX(market_week), MIN(market_week), MAX(market_week + 0) FROM market_date_info;
```

Q135. Find the first and last market date on record, the number of calendar days between them and the number of market days held.

Q136. What will this return?

```
SELECT COUNT(*), SUM(quantity), AVG(quantity) FROM customer_purchases WHERE customer_id = 999;
```

Q137. How many distinct (product, price) combinations were sold? List them.

Q138. In one query, compute the average spend per **visit**: total revenue ÷ number of distinct (customer, date) pairs.

Section 12: GROUP BY & HAVING

Q139. Find the revenue for each market date, with the highest-revenue day first.

Q140. Show the revenue and distinct customers for each month, with the month labelled as **'Jul-2019'** and sorted in date order (not alphabetically).

Q141. Find the customers who spent more than $200 in total.

Q142. Find the customers who visited on at least 10 distinct dates but spent less than $200 in total.

Q143. For each customer, show the first purchase date, the last purchase date and the number of days between them (their tenure).

Q144. Find the customers who bought in 2019 but **never** in 2020, using a single GROUP BY with no subquery.

Q145. Find the customers who bought only **one** of the two products.

Q146. Find the cases where a customer bought the same product more than once on the same market date. Show the count and the transaction times (hint: GROUP_CONCAT).

Q147. Booth-conflict audit: find the (booth, date) pairs with **more than one** vendor assigned. How many are there?

Q148. Find the vendors assigned to **more than one booth** on the same date. List the booths as a comma-separated string.

Q149. Find the (year, week) groups whose market days fall in **different seasons**.

Q150. Every market week should have 2 market days (Wednesday and Saturday). Find the weeks that don't.

Q151. Find the revenue and line count for each hour of the day, split by market weekday. Which hour is busiest on Saturdays?

Q152. Why does this fail in MySQL? Rewrite it correctly and explain why WHERE is the better place for this filter.

```
SELECT customer_id, SUM(quantity * cost_to_customer_per_qty) AS spend
FROM customer_purchases
GROUP BY customer_id
HAVING market_date >= '2020-01-01';
```

Q153. This query ran on MySQL 5.6 but fails on MySQL 8 with error 1055. Explain why, and give two correct fixes.

```
SELECT customer_id, market_date, SUM(quantity) FROM customer_purchases GROUP BY customer_id;
```

Q154. Group the purchase lines into $5 buckets ('0-5', '5-10', ...) and show the count and revenue for each bucket, sorted by bucket.

Section 13: ORDER BY, LIMIT, OFFSET

Q155. Find the top 5 customers by total spend.

Q156. The customer list is shown 10 per page, sorted by last name and then first name. Write the query for **page 3**. Which customers appear, and how many?

Q157. Find the **second highest** distinct line total using LIMIT/OFFSET.

Q158. What do `LIMIT 1, 1`, `LIMIT 1 OFFSET 1` and `LIMIT 1, 3` each return?

Q159. Sort `market_date_info` rows by the market's **real** start time, then by date.

Q160. Find the top 3 purchases by quantity, including every row that ties with the 3rd-place value. Why can plain `LIMIT 3` be misleading?

Q161. Explain `GROUP BY 1 ORDER BY 2 DESC`. When is ordering by column position risky?

Q162. Why does `ORDER BY market_week` give 10, 11, ..., 52, 9? Sort the weeks correctly.

Section 14: Order of Execution

Q163. Write the **logical order** in which MySQL processes FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY and LIMIT. Explain it using a query that finds 2020 customers with more than $100 of spend.

Q164. Why does `WHERE SUM(quantity) > 10` fail? Fix it.

Q165. Does this run in MySQL? Would it run in PostgreSQL or SQL Server?

```
SELECT customer_id, SUM(quantity * cost_to_customer_per_qty) AS spend
FROM customer_purchases GROUP BY customer_id HAVING spend > 200;
```

Q166. Does `GROUP BY yr` work when `yr` is an alias for `YEAR(market_date)`? Is it portable?

Q167. Why can ORDER BY use a SELECT alias when WHERE cannot?

Q168. Walk through this query in execution order and describe the result. Is the output deterministic?

```
SELECT customer_id, COUNT(*) AS n
FROM customer_purchases
WHERE product_id = 2
GROUP BY customer_id
HAVING n >= 5
ORDER BY n DESC
LIMIT 2;
```

Q169. "Average of the 10 largest purchase lines". Why is this query wrong? Fix it.

```
SELECT AVG(quantity * cost_to_customer_per_qty)
FROM customer_purchases
ORDER BY quantity * cost_to_customer_per_qty DESC
LIMIT 10;
```

Q170. What does each of these return?

```
SELECT COUNT(*) FROM customer_purchases WHERE 1 = 0;
SELECT COUNT(*) FROM customer_purchases WHERE 1 = 0 GROUP BY customer_id;
```

Section 15: Subqueries

Q171. Find the purchase lines whose value is above the average line value. How many are there?

Q172. Find the customers who have **never** made a purchase, first with NOT IN and then with NOT EXISTS. What does the result tell you about this extract?

Q173. Without a JOIN, show the **names** of customers who spent more than $200.

Q174. Find the names of the products that have been sold at least once.

Q175. For each purchase line, show the customer's **own** average line value next to it, plus the difference. Use a correlated subquery.

Q176. Find the purchases where the price paid was lower than that day's `original_price` in `vendor_inventory`.

Q177. For each product, find the market date with the highest revenue.

Q178. Find the customer with the highest total spend **without** LIMIT, so that ties are returned too.

Q179. Find the market dates whose revenue was above the average **daily** revenue.

Q180. For product 1, compute the sell-through % on each market date: quantity sold ÷ quantity brought. Get the quantity brought with a correlated subquery on `vendor_inventory`.

Q181. Using EXISTS, find the vendors with inventory records for product 4 in 2020.

Q182. Show each customer's share of total revenue as a percentage.

Q183. Find the market date(s) with the most distinct customers, including ties.

Q184. Find the customer with the **second highest** total spend without LIMIT or OFFSET.

Section 16: Statistics

Q185. For purchase quantity, compute the mean, population standard deviation, sample standard deviation and variance. What is the difference between `STD()`, `STDDEV_POP()` and `STDDEV_SAMP()`?

Q186. Find the **median** purchase quantity using subqueries. Make sure it works for both odd and even row counts and for ties.

Q187. Find the **mode** of purchase quantity, including all values that tie.

Q188. Using z-scores, find the outlier purchase lines whose value is more than 2 standard deviations above the mean.

Q189. For each product in `vendor_inventory`, compute the coefficient of variation (stddev ÷ mean) of the daily quantity brought. Which product has the least predictable supply?

Q190. For each season, show the number of recorded days, the mean, the min, the max, the range and the mid-range of `market_max_temp`.

Q191. Compute the Pearson correlation between `market_min_temp` and `market_max_temp` using only aggregate functions.

Q192. Find the **top 10%** of purchase lines by value using a correlated subquery.

Q193. For each market date, show that day's revenue, the **previous market day's** revenue and the % change. Use correlated subqueries.

Q194. Compute a running (cumulative) total of revenue by market date using a correlated subquery.

Section 17: Interview Logic Scenarios

Q195. Find the customers who shopped on **both** the Wednesday and the Saturday of the same market week at least once. How many such weeks did each customer have?

Q196. What percentage of customers are **repeat** customers (purchases on more than one market date)? And what percentage made purchases in both 2019 and 2020?

Q197. Find the customers whose 2020 spend was higher than their 2019 spend. Show both amounts and the growth %.

Q198. Find the market dates when vendor 7 brought Habanero Peppers (product 1) but **sold none**.

Q199. Build a year-over-year table for July, August and September: one row per month, with columns for 2019 revenue, 2020 revenue and YoY growth %.

Q200. Build a one-row **executive summary** using scalar subqueries only: total revenue, total customers, best customer id, best market date (by revenue) and busiest hour (by line count).

ANSWERS

> All answers are for MySQL 8 (Workbench). Run `USE farmers_market;` first. In this key, `amount` is short for `quantity * cost_to_customer_per_qty`. The numbers quoted come from the CSV extract in this folder.

Section 1: SELECT, Derived Columns & Aliases

A1.

```
SELECT market_date, customer_id, product_id, quantity, cost_to_customer_per_qty,
       ROUND(quantity * cost_to_customer_per_qty, 2) AS line_total
FROM customer_purchases;
```

A2.

```
SELECT customer_id, market_date,
       ROUND(quantity * cost_to_customer_per_qty, 2)         AS line_total,
       ROUND(quantity * cost_to_customer_per_qty * 0.053, 2) AS tax_amount,
       ROUND(quantity * cost_to_customer_per_qty * 1.053, 2) AS amount_payable
FROM customer_purchases;
```

Each column is rounded on its own. For example, 1.004 rounds to 1.00 and 0.004 rounds to 0.00, which add up to 1.00, but 1.008 rounds to 1.01. Round only the final figure, or compute the total from the rounded parts, and be consistent about which one you use.

A3.

```
SELECT market_date, vendor_id, product_id, quantity, original_price,
       quantity * original_price AS stock_value
FROM vendor_inventory
ORDER BY stock_value DESC
LIMIT 10;
```

A4.

```
SELECT market_date, market_min_temp, market_max_temp,
       market_max_temp - market_min_temp AS temp_swing,
       ROUND(((market_min_temp + market_max_temp) / 2 - 32) * 5 / 9, 1) AS avg_temp_c
FROM market_date_info;
```

It works because MySQL converts a VARCHAR to a number whenever the string is used in arithmetic. On days with no reading, the result is NULL, because any arithmetic involving NULL gives NULL.

A5.

```
SELECT customer_id, product_id, quantity,
       ROUND(quantity * 0.4536, 3)                  AS qty_kg,
       ROUND(cost_to_customer_per_qty / 0.4536, 2)  AS price_per_kg
FROM customer_purchases
WHERE product_id IN (1, 2);
```

The price per kg is the price per lb **divided** by 0.4536, not multiplied.

A6. `a = 3.5000`, `b = 3`, `c = 1`, `d = -3` (DIV truncates toward zero), `e = -1` (the sign follows the dividend), `f = 2`. For `f`, MySQL reads '1 lb' as 1 and raises warning 1292.

A7. It fails with **Error 1054: Unknown column 'line_total' in 'where clause'**, because WHERE runs before SELECT and the alias doesn't exist yet. There are two fixes:

```
SELECT quantity * cost_to_customer_per_qty AS line_total
FROM customer_purchases
WHERE quantity * cost_to_customer_per_qty > 30;

-- or
SELECT * FROM (SELECT quantity * cost_to_customer_per_qty AS line_total FROM
customer_purchases) t
WHERE line_total > 30;
```

A8.

```
SELECT customer_id, market_date,
       ROUND(quantity * cost_to_customer_per_qty, 2) AS line_total,
       FLOOR(quantity * cost_to_customer_per_qty)    AS reward_points   -- TRUNCATE(x, 0) also
works
FROM customer_purchases;
```

A9.

```
SELECT DISTINCT product_id, original_price,
       CEIL(original_price * 1.125 * 20) / 20 AS new_price
FROM vendor_inventory;
```

Multiplying by 20 counts in nickels, CEIL rounds up to a whole nickel, and dividing by 20 converts back to dollars.

A10.

```
SELECT 'Harrisonburg Farmers Market' AS market_name, 2019 AS onboarding_year, vendor_name
FROM vendor;
```

A11. `7 | 7.5 | NULL | 0`. Division by zero gives NULL in MySQL. 'Apple Pie' + 0 gives 0 with a warning, because the string has no leading digits.

A12.

```
SELECT customer_id AS `Customer ID`,
       DAYNAME(market_date) AS `Market Day`,
       ROUND(quantity * cost_to_customer_per_qty, 2) AS `Amount ($)`
FROM customer_purchases
ORDER BY `Amount ($)` DESC;
```

Watch out for `ORDER BY "Amount ($)"`. In MySQL, double quotes make a **string literal**, so that query sorts by a constant and nothing gets sorted. Use backticks.

Section 2: WHERE, AND / OR / NOT

A13.

```
SELECT * FROM market_date_info
WHERE market_day = 'Wednesday' AND market_year = '2019' AND market_rain_flag = 1;
```

This returns 2019-03-20, 07-31, 12-04 and 12-11. `market_year = 2019` also works, because MySQL converts the string to a number, but the conversion happens on every row and an index can't be used.

A14. AND binds tighter than OR, so the query really means `(Saturday AND rain) OR snow`, which includes snow on **any** weekday. The only snow day, 2019-03-09, happens to be a Saturday, which is why the output looks right.

```
SELECT * FROM market_date_info
WHERE market_day = 'Saturday' AND (market_rain_flag = 1 OR market_snow_flag = 1);
```

A15.

```
SELECT * FROM customer_purchases
WHERE product_id = 2 AND cost_to_customer_per_qty <> 3.49;
```

This returns 2 rows: customers 2 and 10 on 2019-07-13, both at 3.00.

A16.

```
SELECT * FROM customer_purchases
WHERE (DAYNAME(market_date) = 'Wednesday' AND transaction_time BETWEEN '18:30:00' AND
'19:00:00')
   OR (DAYNAME(market_date) = 'Saturday'  AND transaction_time BETWEEN '13:30:00' AND
'14:00:00');
```

A17.

```
SELECT * FROM customer_purchases
WHERE quantity = FLOOR(quantity);        -- or: MOD(quantity, 1) = 0
```

This returns 3 rows, with quantities 2, 1 and 3.

A18.

```
SELECT * FROM market_date_info WHERE NOT (market_rain_flag = 1 OR market_snow_flag = 1);
SELECT * FROM market_date_info WHERE market_rain_flag <> 1 AND market_snow_flag <> 1;   -- De
Morgan
```

A19. It returns **29**. Only 31 days have a temperature, and 2 of them are 50. For the other 97 days, `NULL <> 50` is NULL (unknown), and WHERE keeps only rows where the condition is TRUE. To also count the NULL days:

```
SELECT COUNT(*) FROM market_date_info
WHERE market_min_temp <> 50 OR market_min_temp IS NULL;          -- 126
-- or: WHERE NOT (market_min_temp <=> 50)
```

A20.

```
SELECT * FROM customer
WHERE customer_zip = '22801' AND customer_last_name < 'N';
```

The result is Connor, Harris, Bullard, Forbes, Edwards, Lopes, Messina, Laurie and Hassan (9 rows). 'Messina' > 'M' because a longer string that begins with the same prefix sorts **after** the prefix, so `<= 'M'` only keeps names that are exactly 'M'.

A21.

```
SELECT * FROM customer_purchases
WHERE quantity * cost_to_customer_per_qty > 20 AND transaction_time < '12:00:00';
```

A22.

```
SELECT * FROM market_date_info WHERE market_rain_flag XOR market_snow_flag;
SELECT * FROM market_date_info WHERE market_rain_flag <> market_snow_flag;
```

Both return 17 rows: the 16 rainy days plus the 1 snow day. No day had both.

A23. It runs and returns 15 customers. Because the column is VARCHAR and the literal is a number, MySQL converts **every** zip to a number. That means an index on `customer_zip` can't be used, and a value such as '022801' would also match. Write `customer_zip = '22801'` instead.

A24. The comparison is between strings, character by character. '8:00 AM', '9:00 AM' and '4:00 PM' all start with a character greater than '1', so all of them are "greater" than '12:00 PM'. The fix is to convert the text to a real TIME:

```
SELECT * FROM market_date_info
WHERE STR_TO_DATE(market_start_time, '%l:%i %p') < '12:00:00';
```

A25.

```
SELECT * FROM product
WHERE product_qty_type <> 'unit' OR product_qty_type IS NULL;    -- 9 rows
-- or: WHERE NOT (product_qty_type <=> 'unit')
```

`NULL <> 'unit'` is unknown, so on its own the condition drops products 14 and 15.

A26. AND is evaluated first, so the query means `(date AND customer 2) OR (customer 10 AND product 2)`. That returns everything customer 2 bought that day, plus customer 10's product 2 purchases on **every** date.

```
SELECT * FROM customer_purchases
WHERE market_date = '2019-07-13' AND customer_id IN (2, 10) AND product_id = 2;
```

Section 3: IN, NOT IN, BETWEEN

A27.

```
SELECT * FROM customer_purchases
WHERE customer_id IN (1, 4, 14) AND DAYNAME(market_date) = 'Saturday';
```

A28.

```
SELECT * FROM vendor WHERE vendor_type IN ('Prepared Foods', 'Eggs & Meats');   -- vendors 1,
8, 9
```

A29.

```
SELECT * FROM booth
WHERE booth_number NOT IN (SELECT booth_number FROM vendor_booth_assignments);  -- 3, 4, 5, 9,
12
```

NOT IN is safe here only because `booth_number` can't be NULL in the subquery. `NOT EXISTS` is safe even with NULLs.

A30. It returns **no rows**. `x NOT IN ('unit', NULL)` means `x <> 'unit' AND x <> NULL`, and the second part is always NULL, so the whole condition is never TRUE.

```
SELECT product_name FROM product
WHERE product_qty_type <> 'unit' OR product_qty_type IS NULL;
```

A31.

```
SELECT * FROM customer_purchases
WHERE market_date BETWEEN '2019-07-01' AND '2019-07-31';
```

For a DATETIME, '2019-07-31' means 2019-07-31 **00:00:00**, so anything later that day is excluded. The safe pattern for a DATETIME is `>= '2019-07-01' AND < '2019-08-01'`.

A32.

```
SELECT * FROM market_date_info WHERE market_min_temp BETWEEN 40 AND 55;
```

With number literals, MySQL compares the values as numbers. With `BETWEEN '40' AND '55'`, it compares them as text: '5' would count as "between" and '100' wouldn't. Every temperature in this data has two digits, so the result happens to be the same, but that won't hold in general.

A33. It returns `0`. `BETWEEN a AND b` means `>= a AND <= b`, and no value is ≥ 5 and ≤ 3 at the same time. Always put the lower bound first.

A34.

```
SELECT * FROM customer_purchases
WHERE DAYNAME(market_date) = 'Saturday'
  AND transaction_time BETWEEN '11:00:00' AND '13:00:00';
```

A35. 'Deanna', 'Dawn', 'Darrell' and 'Duane' are all greater than 'D', so `BETWEEN 'B' AND 'D'` excludes them.

```
SELECT * FROM customer
WHERE customer_first_name >= 'B' AND customer_first_name < 'E';
-- or: WHERE LEFT(customer_first_name, 1) BETWEEN 'B' AND 'D'
```

A36.

```
SELECT * FROM product
WHERE product_category_id IN (
    SELECT product_category_id FROM product_category
    WHERE product_category_name IN ('Fresh Fruits & Vegetables', 'Eggs & Meat (Fresh or
Frozen)'));
```

A37.

```
SELECT * FROM vendor
WHERE vendor_id NOT IN (SELECT DISTINCT vendor_id FROM vendor_inventory);   -- 1,2,3,4,5,6,9
```

A38.

```
SELECT * FROM customer_purchases
WHERE market_date NOT BETWEEN '2019-06-01' AND '2019-10-30';
```

Expect **93 rows**, all from 2020, because every 2019 purchase falls between July and September. This is a quick way to sanity-check a date filter.

Section 4: LIKE

A39.

```
SELECT * FROM product WHERE product_name LIKE '%organic%';     -- 1, 2, 3, 18, 21
```

It matches 'Organic' because the default collation (`utf8mb4_0900_ai_ci`) is **c**ase-**i**nsensitive. For a case-sensitive match, use `product_name LIKE '%organic%' COLLATE utf8mb4_bin`, which returns 0 rows.

A40.

```
SELECT * FROM product WHERE product_name LIKE '%Jar' OR product_name LIKE '%Bag';
```

This returns 4 rows: 4, 12, 23 **and 19**, because 'Farmers Market Resuable Shopping Bag' also ends in 'Bag'. If you only wanted the packaging variants, use `LIKE '% - Jar' OR LIKE '% - Bag'`.

A41.

```
SELECT * FROM vendor WHERE vendor_name LIKE '%&%';                  -- 1, 2, 6
SELECT DISTINCT vendor_type FROM vendor WHERE vendor_type LIKE '%:%';
```

A42.

```
SELECT * FROM customer WHERE customer_first_name LIKE '___';        -- Bob, Ada, Iva
```

A43.

```
SELECT * FROM customer WHERE customer_last_name LIKE '_a%';
-- Washington, Harris, Valenzuela, Paulson, Rai, Laurie, Nale, Hassan
```

A44.

```
SELECT * FROM product WHERE product_name LIKE '%\_%';              -- backslash is the default
escape
SELECT * FROM product WHERE product_name LIKE '%!_%' ESCAPE '!';    -- custom escape character
```

Both return 0 rows. Without the escape, `'%_%'` matches every non-empty name.

A45.

```
SELECT * FROM vendor WHERE vendor_name LIKE CONCAT(vendor_owner_first_name, '%');   -- 1, 7, 8
```

A46.

```
SELECT * FROM vendor WHERE vendor_name LIKE CONCAT('%', vendor_owner_last_name, '%');  --
vendor 2
-- or: WHERE INSTR(vendor_name, vendor_owner_last_name) > 0
```

A47.

```
SELECT * FROM booth WHERE booth_description LIKE '%  %';     -- 5, 6, 8, 9, 10, 11, 12
```

A48. `=` returns 1 row (product 14) and `LIKE 'Red Potatoes%'` returns 2 rows (14 and 15). The request is for *products* of that type, so LIKE is the right choice.

A49.

```
SELECT * FROM product WHERE LEFT(product_size, 1) BETWEEN '0' AND '9';
SELECT * FROM product WHERE product_size REGEXP '^[0-9]';
```

Both return products 4, 5, 7, 8, 10, 11, 12, 13, 20 and 23. NULL and ' ' are excluded automatically.

A50.

```
SELECT * FROM customer
WHERE customer_last_name LIKE CONCAT(LEFT(customer_first_name, 1), '%');     -- Betty Bullard
```

Section 5: CONCAT & Formatting

A51.

```
SELECT CONCAT(customer_last_name, ', ', customer_first_name) AS display_name
FROM customer
ORDER BY customer_last_name, customer_first_name;
```

A52.

```
SELECT customer_id,
       LOWER(CONCAT(LEFT(customer_first_name, 1), customer_last_name, customer_id,
'@fmarket.com')) AS email
FROM customer;
```

A53. The basic `CONCAT(product_name, ' (', product_size, ', ', product_qty_type, ')')` returns **NULL** for 14 and 15, because their qty type is NULL. The version below never returns NULL and leaves out the brackets when there's nothing to put in them:

```
SELECT product_id,
       CONCAT(product_name,
              IFNULL(CONCAT(' (', NULLIF(CONCAT_WS(', ', NULLIF(TRIM(product_size), ''),
product_qty_type), ''), ')'), '')
       ) AS product_label
FROM product;
```

It works because CONCAT_WS skips NULLs, NULLIF turns an empty result into NULL, and CONCAT with a NULL returns NULL, which IFNULL then replaces with ''.

A54. `'Booth-7' | NULL | 'A-B' | NULL`. CONCAT_WS returns NULL only when the **separator** is NULL.

A55.

```
SELECT booth_number, CONCAT(booth_price_level, '-', LPAD(booth_number, 2, '0')) AS booth_code
FROM booth;
```

A56.

```
SELECT market_date,
       CONCAT(DATE_FORMAT(market_date, '%a, %d-%b-%Y'), ' (Wk ', LPAD(market_week, 2, '0'),
')') AS label
FROM market_date_info;
```

A57.

```
SELECT CONCAT('Cust ', customer_id, ' bought ', quantity, ' lbs @ $', cost_to_customer_per_qty,
              ' = $', ROUND(quantity * cost_to_customer_per_qty, 2)) AS invoice_line
FROM customer_purchases;
```

Because the columns are DECIMAL(16,2), a quantity of 2 prints as '2.00'.

A58.

```
SELECT customer_id,
       CONCAT(LEFT(customer_last_name, 1), REPEAT('*', CHAR_LENGTH(customer_last_name) - 1)) AS
masked
FROM customer;
```

A59.

```
SELECT vendor_name,
       LOWER(REPLACE(REPLACE(vendor_name, ' & ', '_and_'), ' ', '_')) AS handle
FROM vendor;
```

The order matters: replace ' & ' **first**, or the spaces around it will already have been turned into underscores. The accented 'á' stays in 'hernández_...'.

A60. It returns `0` with warnings. In MySQL, `||` is **logical OR** by default. 'Eggs' and 'Meat' both convert to 0, and 0 OR 0 is 0. Use `CONCAT('Eggs', 'Meat')`, or turn on `sql_mode = PIPES_AS_CONCAT`.

Section 6: INSTR, LOCATE, LEFT, RIGHT, SUBSTRING, SUBSTRING_INDEX

A61.

```
SELECT product_name,
       CASE WHEN INSTR(product_name, ' - ') > 0
            THEN LEFT(product_name, INSTR(product_name, ' - ') - 1)
            ELSE product_name END                                    AS base_name,
       CASE WHEN INSTR(product_name, ' - ') > 0
            THEN SUBSTRING(product_name, INSTR(product_name, ' - ') + 3) END AS variant
FROM product;
```

A62.

```
SELECT product_name,
       SUBSTRING_INDEX(product_name, ' - ', 1) AS base_name,
       CASE WHEN INSTR(product_name, ' - ') > 0 THEN SUBSTRING_INDEX(product_name, ' - ', -1)
END AS variant
FROM product;
```

When the delimiter isn't in the string, SUBSTRING_INDEX returns the **whole string**, so without the CASE, 'Eggs' would get the variant 'Eggs'.

A63.

```
SELECT product_name, SUBSTRING_INDEX(product_name, ' ', 1) AS first_word FROM product;
```

For 'Eggs', INSTR returns 0, so the expression becomes `LEFT(name, -1)`, which returns '' (an empty string).

A64.

```
SELECT product_name,
       SUBSTRING_INDEX(product_name, ' ', -1)                    AS last_word_v1,
       CASE WHEN INSTR(product_name, ' ') = 0 THEN product_name
            ELSE RIGHT(product_name, INSTR(REVERSE(product_name), ' ') - 1) END AS last_word_v2
FROM product;
```

A65.

```
SELECT booth_number,
       CASE WHEN INSTR(booth_description, '-') > 0
            THEN TRIM(SUBSTRING(booth_description, INSTR(booth_description, '-') + 1)) END AS
table_size
FROM booth;
```

Splitting on '-' and then trimming handles both ' - ' and ' - '.

A66.

```
SELECT SUBSTRING_INDEX(booth_description, ' ', 1) AS first_word, COUNT(*) AS booths
FROM booth
GROUP BY first_word;          -- First 5, Second 2, Third 2, Back 3
```

A67.

```
SELECT market_start_time,
       CAST(SUBSTRING_INDEX(market_start_time, ':', 1) AS UNSIGNED)          AS hour_12,
       RIGHT(market_start_time, 2)                                           AS am_pm
FROM market_date_info;
-- without SUBSTRING_INDEX: LEFT(market_start_time, INSTR(market_start_time, ':') - 1)
```

A68.

```
SELECT market_date, market_start_time, market_end_time,
       end_24 - start_24 AS hours_open
FROM (
  SELECT market_date, market_start_time, market_end_time,
         LEFT(market_start_time, INSTR(market_start_time, ':') - 1) % 12
           + CASE WHEN RIGHT(market_start_time, 2) = 'PM' THEN 12 ELSE 0 END   AS start_24,
         LEFT(market_end_time, INSTR(market_end_time, ':') - 1) % 12
           + CASE WHEN RIGHT(market_end_time, 2) = 'PM' THEN 12 ELSE 0 END     AS end_24
  FROM market_date_info
) t
ORDER BY hours_open, market_date;
```

The `% 12` turns 12 AM into 0 and keeps 12 PM correct. Wednesdays are open 3 hours and Saturdays 6. On **2019-03-09** (the snow day), the market opened at 9 AM and was open only 5 hours.

A69.

```
SELECT product_id, product_size,
       LEFT(product_size, INSTR(product_size, ' ') - 1)  AS size_value,
       SUBSTRING(product_size, INSTR(product_size, ' ') + 1) AS size_unit
FROM product
WHERE INSTR(product_size, ' ') > 0 AND LEFT(product_size, 1) BETWEEN '0' AND '9';
```

This returns products 4, 5, 10, 11, 12, 13 and 23. The value '1/2' (product 12) is text, not a number.

A70. `'' | 'Pie' | '' | 'Pie' | 'Pie' | 'ppl'`. SUBSTRING positions start at 1, so position 0 returns an empty string. A negative start counts from the end.

A71. `2 | 2 | 7 | 0 | 7`. The third result is 7 because the search starts at position 4 and, under the case-insensitive collation, 'P' in 'Pie' counts as a match.

A72.

```
SELECT product_name,
       LOCATE(' ', product_name, INSTR(product_name, ' ') + 1) AS second_space_pos
FROM product
WHERE CHAR_LENGTH(product_name) - CHAR_LENGTH(REPLACE(product_name, ' ', '')) >= 2;
```

A73. The ' - ' separator needs care. 'Carrots - Organic' has exactly two spaces, and its "middle word" would be '-'. Replace ' - ' with a single space first:

```
SELECT product_name,
       SUBSTRING_INDEX(SUBSTRING_INDEX(clean_name, ' ', 2), ' ', -1) AS middle_word
FROM (SELECT product_name, REPLACE(product_name, ' - ', ' ') AS clean_name FROM product) t
WHERE CHAR_LENGTH(clean_name) - CHAR_LENGTH(REPLACE(clean_name, ' ', '')) = 2;
```

A74.

```
SELECT vendor_type, SUBSTRING_INDEX(vendor_type, ':', 1) AS type_group FROM vendor;
```

No CASE is needed here. When there's no ':', SUBSTRING_INDEX returns the whole string, which is exactly what's wanted.

A75.

```
SELECT transaction_time,
       LEFT(transaction_time, 2)        AS hh_str,  SUBSTRING(transaction_time, 4, 2) AS
mm_str,
       HOUR(transaction_time)           AS hh,      MINUTE(transaction_time)          AS mm
FROM customer_purchases;
```

String functions implicitly convert a TIME to 'HH:MM:SS'. The string version breaks for values of 100 hours or more, and for negative times.

A76.

```
SELECT market_date, LEFT(market_date, 4) AS y_str, SUBSTRING(market_date, 6, 2) AS m_str,
       YEAR(market_date) AS y, MONTH(market_date) AS m
FROM customer_purchases;
```

YEAR() and MONTH() are safer. They return integers and don't depend on how the date is formatted as text. Note that a filter such as `YEAR(market_date) = 2019` can't use an index. Prefer a range like `market_date >= '2019-01-01' AND market_date < '2020-01-01'`.

A77.

```
SELECT * FROM product
WHERE INSTR(product_name, ' - ') > 0
  AND SUBSTRING_INDEX(product_name, ' - ', -1) NOT IN ('Organic', 'Jar', 'Bag');   -- 15 'Red
Potatoes - Small'
```

A78.

```
SELECT customer_id,
       CONCAT(LEFT(customer_first_name, 1), LEFT(customer_last_name, 1), '-',
              RIGHT(customer_zip, 3), '-', LPAD(customer_id, 2, '0')) AS customer_code
FROM customer;           -- JC-801-01, MD-821-02, ...
```

Section 7: TRIM, LENGTH, CHAR_LENGTH, REPLACE, REVERSE, UPPER/LOWER

A79. `10 | 9 | 1 | 0 | NULL`. In utf8mb4, 'á' takes 2 bytes: LENGTH counts bytes and CHAR_LENGTH counts characters.

A80.

```
SELECT * FROM vendor
WHERE LENGTH(vendor_name) <> CHAR_LENGTH(vendor_name)
   OR LENGTH(vendor_owner_last_name) <> CHAR_LENGTH(vendor_owner_last_name);    -- vendor 2
```

A81.

```
SELECT booth_number,
       CHAR_LENGTH(booth_description)                          AS old_len,
       CHAR_LENGTH(REPLACE(booth_description, '  ', ' '))      AS new_len,
       REPLACE(booth_description, '  ', ' ')                   AS cleaned
FROM booth
WHERE booth_description LIKE '%  %';
```

Each row gets 1 character shorter. With three or more spaces in a row, you'd need to apply REPLACE again.

A82.

```
SELECT product_name,
       CHAR_LENGTH(clean) - CHAR_LENGTH(REPLACE(clean, ' ', '')) + 1 AS word_count
FROM (SELECT product_name, REPLACE(product_name, ' - ', ' ') AS clean FROM product) t;
```

A83.

```
SELECT customer_id,
       CHAR_LENGTH(CONCAT(customer_first_name, customer_last_name))
     - CHAR_LENGTH(REPLACE(LOWER(CONCAT(customer_first_name, customer_last_name)), 'e', '')) AS
e_count
FROM customer;
```

REPLACE is case-sensitive even under a _ci collation, so without LOWER the capital 'E' in 'Edwards' wouldn't be counted.

A84.

```
SELECT * FROM customer
WHERE LOWER(customer_first_name) = LOWER(REVERSE(customer_first_name));    -- Bob, Ada
```

Under a _ci collation, `customer_first_name = REVERSE(customer_first_name)` also works, but LOWER makes the intent explicit.

A85. `'Eggs' | '120' | 'Pepper' | 'Jar' | 'a ' | ' a'`.

A86.

```
SELECT product_size,
       CONCAT(UPPER(LEFT(product_size, 1)), LOWER(SUBSTRING(product_size, 2))) AS size_clean
FROM product;
```

A87.

```
SELECT * FROM product WHERE product_size IS NOT NULL AND TRIM(product_size) = '';   -- product
15
```

Whether `' ' = ''` is true depends on the collation's **pad attribute**. MySQL 8's default `utf8mb4_0900_ai_ci` is NO PAD, so `' ' = ''` is FALSE there. Older collations such as `utf8_general_ci` and `latin1` (the MySQL 5.6 defaults) are PAD SPACE and ignore trailing spaces, so there it's TRUE.

A88.

```
SELECT * FROM vendor WHERE vendor_name LIKE '%PIES%';                          -- Annies Pies
SELECT * FROM vendor WHERE vendor_name LIKE '%PIES%' COLLATE utf8mb4_bin;      -- 0 rows
```

The binary collation compares exact bytes, and 'Pies' ≠ 'PIES'. `LIKE BINARY` also works, but it's deprecated in MySQL 8.0.

A89.

```
SELECT product_name, CHAR_LENGTH(product_name) AS len
FROM product
WHERE CHAR_LENGTH(product_name) = (SELECT MAX(CHAR_LENGTH(product_name)) FROM product);
-- 'Farmers Market Resuable Shopping Bag' (36)
```

A90.

```
SELECT vendor_name,
       CASE WHEN INSTR(vendor_name, 'e') = 0 THEN 0
            ELSE CHAR_LENGTH(vendor_name) - INSTR(REVERSE(vendor_name), 'e') + 1 END AS
last_e_pos
FROM vendor;
```

INSTR is case-insensitive here, so it also finds an 'E'.

Section 8: NULL Handling

A91.

```
SELECT market_date FROM market_date_info
WHERE market_min_temp IS NULL AND market_max_temp IS NULL;      -- 97 days
```

`= NULL` evaluates to NULL, never TRUE, so no rows pass the filter.

A92.

```
SELECT market_date,
       COALESCE(market_min_temp, 'Not recorded') AS min_temp,
       COALESCE(market_max_temp, 'Not recorded') AS max_temp
FROM market_date_info;
```

If the columns were numeric, COALESCE would turn the whole column into text, which changes how it sorts.

A93. `NULL | 1 | 1 | NULL | 'x' | 0 | NULL | 5 | NULL`.

A94. `128 | 31 | 0`. `COUNT(*)` counts rows, and `COUNT(col)` counts only the non-NULL values. `special_notes` is NULL on every row.

A95.

```
SELECT AVG(market_min_temp)               AS correct_avg,   -- 54.32 (over 31 recorded days)
       SUM(market_min_temp) / COUNT(*)    AS wrong_1,       -- 13.16 (1684 / 128)
       AVG(COALESCE(market_min_temp, 0))  AS wrong_2        -- 13.16
FROM market_date_info;
```

A missing reading is **unknown**, not 0 °F, so AVG, which skips NULLs, gives the right answer.

A96.

```
SELECT product_id, COALESCE(NULLIF(TRIM(product_size), ''), 'Size not specified') AS size_label
FROM product;
```

A97. In MySQL, `10/0` returns **NULL** with a warning. SQL Server and PostgreSQL raise an error instead. To avoid division by zero anywhere:

```
SELECT market_date,
       SUM(quantity * cost_to_customer_per_qty) / NULLIF(COUNT(DISTINCT customer_id), 0) AS
rev_per_customer
FROM customer_purchases
GROUP BY market_date;
```

A98.

```
SELECT * FROM product
WHERE NULLIF(TRIM(product_size), '') IS NULL
   OR NULLIF(TRIM(product_qty_type), '') IS NULL;       -- 14, 15
```

A99. `14`, not 16. The 2 NULL rows make the condition unknown, so they're left out.

A100. With ASC, MySQL puts NULLs **first**.

```
SELECT market_date, market_min_temp
FROM market_date_info
ORDER BY market_min_temp IS NULL, market_min_temp + 0;
```

`IS NULL` is 0 for rows with a value and 1 for NULLs, so the NULLs go last. The `+ 0` sorts the text as numbers.

A101.

```
SELECT market_date, IF(market_rain_flag = 1, 'Rain', 'Dry') AS rain_status FROM
market_date_info;
```

IFNULL takes exactly 2 arguments and is MySQL-only. COALESCE takes any number of arguments and is standard SQL.

A102. `42 | NULL | NULL | NULL`. There are 42 market days in 2020 but no temperatures, so SUM returns **NULL**, not 0. Use `COALESCE(SUM(...), 0)` if you need 0.

Section 9: CASE and SUM(CASE)

A103.

```
SELECT market_date, market_max_temp,
       CASE WHEN market_max_temp IS NULL THEN 'Unknown'
            WHEN market_max_temp < 50   THEN 'Cold'
            WHEN market_max_temp < 70   THEN 'Mild'
            ELSE 'Warm' END AS temp_band
FROM market_date_info;
```

A104. A CASE stops at the **first** WHEN that matches, and every temperature ≥ 70 is also ≥ 50. Test the highest threshold first:

```
CASE WHEN market_max_temp >= 70 THEN 'Warm'
     WHEN market_max_temp >= 50 THEN 'Mild'
     ELSE 'Cold' END
```

This version still labels NULL temperatures as 'Cold'. Add `WHEN market_max_temp IS NULL THEN 'Unknown'` first.

A105.

```
SELECT CASE WHEN quantity * cost_to_customer_per_qty < 5   THEN 'Small'
            WHEN quantity * cost_to_customer_per_qty <= 20 THEN 'Medium'
            ELSE 'Large' END AS size_band,
       COUNT(*) AS line_count,
       ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS revenue
FROM customer_purchases
GROUP BY size_band;
```

A106.

```
SELECT product_id,
       ROUND(SUM(CASE WHEN YEAR(market_date) = 2019 THEN quantity * cost_to_customer_per_qty
ELSE 0 END), 2) AS sales_2019,
       ROUND(SUM(CASE WHEN YEAR(market_date) = 2020 THEN quantity * cost_to_customer_per_qty
ELSE 0 END), 2) AS sales_2020
FROM customer_purchases
GROUP BY product_id;
-- product 1: 1224.51 | 1580.51     product 2: 1061.17 | 0 (the extract has no 2020 rows for
product 2)
```

A107.

```
SELECT customer_id,
       SUM(CASE WHEN DAYNAME(market_date) = 'Wednesday' THEN 1 ELSE 0 END) AS wed_lines,
       SUM(CASE WHEN DAYNAME(market_date) = 'Saturday'  THEN 1 ELSE 0 END) AS sat_lines
FROM customer_purchases
GROUP BY customer_id;
```

A108.

```
SELECT ROUND(100 * SUM(CASE WHEN transaction_time < '12:00:00' THEN 1 ELSE 0 END) / COUNT(*),
2) AS pct_v1,
       ROUND(100 * AVG(transaction_time < '12:00:00'), 2)                                AS
pct_v2
FROM customer_purchases;
```

In MySQL a comparison evaluates to 1 or 0, so AVG of the comparison gives the fraction of rows where it's true.

A109. `'not equal' | 'no match' | NULL`. A simple CASE compares with `=`, so NULL never matches NULL. A CASE with no ELSE returns NULL when nothing matches.

A110.

```
SELECT market_season FROM market_date_info
GROUP BY market_season
ORDER BY CASE market_season WHEN 'Spring' THEN 1 WHEN 'Summer/Early Fall' THEN 2 ELSE 3 END;

SELECT market_season FROM market_date_info
GROUP BY market_season
ORDER BY FIELD(market_season, 'Spring', 'Summer/Early Fall', 'Late Fall/Holiday');
```

A111.

```
SELECT market_date,
       CASE WHEN market_rain_flag = 1 AND market_snow_flag = 1 THEN 'Rain & Snow'
            WHEN market_rain_flag = 1 THEN 'Rain'
            WHEN market_snow_flag = 1 THEN 'Snow'
            ELSE 'Clear' END AS weather
FROM market_date_info;
```

The combined condition must come first. Otherwise a day with both rain and snow would stop at 'Rain'.

A112.

```
SELECT market_season,
       ROUND(100 * SUM(market_rain_flag) / COUNT(*), 1) AS rain_pct
FROM market_date_info
GROUP BY market_season;       -- Spring 17.6, Summer/Early Fall 8.3, Late Fall/Holiday 11.8
```

A113.

```
SELECT SUM(CASE WHEN booth_price_level = 'A' THEN 1 ELSE 0 END) AS level_a,   -- 3
       SUM(CASE WHEN booth_price_level = 'B' THEN 1 ELSE 0 END) AS level_b,   -- 4
       SUM(CASE WHEN booth_price_level = 'C' THEN 1 ELSE 0 END) AS level_c    -- 5
FROM booth;
```

A114.

```
SELECT customer_id, market_date, cost_to_customer_per_qty,
       CASE WHEN cost_to_customer_per_qty <
                 (SELECT MAX(cost_to_customer_per_qty) FROM customer_purchases WHERE product_id
= 2)
            THEN 'Discounted' ELSE 'Full price' END AS price_flag
FROM customer_purchases
WHERE product_id = 2;
```

A115.

```
SELECT * FROM customer_purchases
WHERE quantity * cost_to_customer_per_qty >
      CASE DAYNAME(market_date) WHEN 'Saturday' THEN 20 WHEN 'Wednesday' THEN 10 END;
```

A116.

```
SELECT product_id, product_name, product_size, product_qty_type,
       CASE WHEN product_qty_type IS NULL                THEN 'Missing qty type'
            WHEN NULLIF(TRIM(product_size), '') IS NULL  THEN 'Missing size'
            WHEN product_size REGEXP '^[0-9]+$'          THEN 'Size is a bare number'
            ELSE 'OK' END AS dq_status
FROM product;
```

Products 14 and 15 get 'Missing qty type'. Both also have a missing size, but the first WHEN that matches wins and hides the second problem. Products 7, 8 and 20 get 'Size is a bare number'.

Section 10: Numeric & Date Functions

A117. `6.99 | 3 | -3 | 1200 | 6.99 | 5 | -5 | 2`. Exact decimals round half **away from zero**. The float literal `2.5e0` rounds half **to even**, giving 2.

A118.

```
SELECT ROUND(quantity * cost_to_customer_per_qty, 2)          AS line_total,
       ROUND(quantity * cost_to_customer_per_qty * 4) / 4     AS cash_total
FROM customer_purchases;
```

A119.

```
SELECT market_date, DAYNAME(market_date) AS dow, MONTHNAME(market_date) AS month_name,
       QUARTER(market_date) AS qtr, WEEK(market_date, 3) AS iso_week, market_week,
       WEEK(market_date) AS week_mode0
FROM market_date_info;

SELECT SUM(market_week <> WEEK(market_date, 3)) AS mismatches FROM market_date_info;    -- 0
```

`market_week` matches the **ISO week**, which is `WEEK(d, 3)` (the same as `WEEKOFYEAR`). The default `WEEK(d)` is mode 0 (weeks start on Sunday) and gives 8 for 2019-03-02, while the stored value is 9.

A120.

```
SELECT customer_id, MAX(market_date) AS last_purchase,
       DATEDIFF('2020-10-10', MAX(market_date)) AS days_since_last
FROM customer_purchases
GROUP BY customer_id
ORDER BY days_since_last DESC;
```

A121.

```
SELECT * FROM customer_purchases WHERE DAY(market_date) <= 7;
```

A122.

```
SELECT market_date,
       STR_TO_DATE(market_start_time, '%l:%i %p')                      AS start_t,
       STR_TO_DATE(market_end_time,   '%l:%i %p')                      AS end_t,
       TIMEDIFF(STR_TO_DATE(market_end_time, '%l:%i %p'),
                STR_TO_DATE(market_start_time, '%l:%i %p'))            AS duration
FROM market_date_info
ORDER BY start_t, market_date;
```

A123.

```
SELECT market_date, transaction_time,
       (TIME_TO_SEC(transaction_time)
        - TIME_TO_SEC(CASE WHEN DAYNAME(market_date) = 'Wednesday' THEN '16:00:00' ELSE
'08:00:00' END)) / 60
       AS minutes_since_open
FROM customer_purchases;
```

A124.

```
SELECT * FROM customer_purchases WHERE DAYOFWEEK(market_date) = 7;
```

DAYOFWEEK numbers the days from **Sunday = 1** to Saturday = 7. By contrast, `WEEKDAY()` numbers them from Monday = 0 to Sunday = 6.

A125.

```
SELECT quantity,
       ROUND(quantity * 16) DIV 16 AS lbs,
       ROUND(quantity * 16) % 16   AS oz
FROM customer_purchases;
```

Work in total ounces first. Otherwise 0.99 lb becomes "0 lb 16 oz" instead of "1 lb 0 oz".

A126. `1 | 1 | -1 | 1024 | 4 | 3.5 | -1`.

Section 11: Aggregates & DISTINCT

A127.

```
SELECT COUNT(*)                                          AS lines,          -- 272
       COUNT(DISTINCT customer_id)                       AS customers,      -- 26
       ROUND(SUM(quantity * cost_to_customer_per_qty), 2)  AS revenue,        -- 3866.18
       ROUND(AVG(quantity * cost_to_customer_per_qty), 2)  AS avg_line,       -- 14.21
       ROUND(MIN(quantity * cost_to_customer_per_qty), 2)  AS min_line,
       ROUND(MAX(quantity * cost_to_customer_per_qty), 2)  AS max_line        -- 39.91
FROM customer_purchases;
```

A128.

```
SELECT COUNT(DISTINCT customer_id, market_date) AS visits FROM customer_purchases;     -- 221
```

A129. `22 | 15 | 23`. The NULL size isn't counted, but the ' ' (space) is a real value and is counted.

A130.

```
SELECT ROUND(AVG(cost_to_customer_per_qty), 4)                                  AS
simple_avg,   -- 3.4800
       ROUND(SUM(quantity * cost_to_customer_per_qty) / SUM(quantity), 4)       AS
weighted_avg  -- 3.4692
FROM customer_purchases WHERE product_id = 2;
```

The weighted average is correct. The two discounted lines were large ones (6.27 and 6.70 lbs), so they deserve more weight than a 0.02 lb line.

A131. It fails with **Error 3065: ORDER BY clause is not in SELECT list … incompatible with DISTINCT**. Each customer has many dates, so MySQL doesn't know which date to sort by. Choose one explicitly:

```
SELECT customer_id FROM customer_purchases
GROUP BY customer_id
ORDER BY MIN(market_date);          -- ordered by first visit
```

A132.

```
SELECT (SELECT COUNT(DISTINCT market_date) FROM vendor_inventory   WHERE vendor_id = 7) AS
days_with_stock,
       (SELECT COUNT(DISTINCT market_date) FROM customer_purchases WHERE vendor_id = 7) AS
days_with_sales;
```

A133.

```
SELECT vendor_id,
       ROUND(SUM(quantity * original_price), 2) AS stock_value,   -- 7: 31851.51, 8: 19029.00
       SUM(quantity)                            AS total_qty
FROM vendor_inventory
GROUP BY vendor_id;
```

`total_qty` adds pounds and units together, so it has no real meaning. The value total is the meaningful figure.

A134. `'9' | '10' | 52`. The column is text, so '9' is greater than '52' because '9' > '5'. Adding `+ 0` makes it numeric.

A135.

```
SELECT MIN(market_date) AS first_day, MAX(market_date) AS last_day,
       DATEDIFF(MAX(market_date), MIN(market_date)) AS days_span,    -- 511
       COUNT(*) AS market_days                                       -- 128
FROM market_date_info;
```

A136. `0 | NULL | NULL`. COUNT of an empty set is 0, and every other aggregate is NULL.

A137.

```
SELECT DISTINCT product_id, cost_to_customer_per_qty FROM customer_purchases;   -- (1,6.99)
(2,3.49) (2,3.00)
SELECT COUNT(DISTINCT product_id, cost_to_customer_per_qty) FROM customer_purchases;   -- 3
```

A138.

```
SELECT ROUND(SUM(quantity * cost_to_customer_per_qty) / COUNT(DISTINCT customer_id,
market_date), 2)
       AS avg_spend_per_visit
FROM customer_purchases;
```

Section 12: GROUP BY & HAVING

A139.

```
SELECT market_date, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS revenue
FROM customer_purchases
GROUP BY market_date
ORDER BY revenue DESC;          -- top: 2019-08-10 at 164.17
```

A140.

```
SELECT DATE_FORMAT(market_date, '%b-%Y') AS month_label,
       ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS revenue,
       COUNT(DISTINCT customer_id) AS customers
FROM customer_purchases
GROUP BY month_label
ORDER BY MIN(market_date);
```

Sorting by `month_label` would be alphabetical ('Aug' before 'Jul'), so sort by `MIN(market_date)`.

A141.

```
SELECT customer_id, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS spend
FROM customer_purchases
GROUP BY customer_id
HAVING SUM(quantity * cost_to_customer_per_qty) > 200
ORDER BY spend DESC;           -- 4, 2, 1, 5
```

A142.

```
SELECT customer_id, COUNT(DISTINCT market_date) AS visits,
       ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS spend
FROM customer_purchases
GROUP BY customer_id
HAVING COUNT(DISTINCT market_date) >= 10 AND SUM(quantity * cost_to_customer_per_qty) < 200;
```

A143.

```
SELECT customer_id, MIN(market_date) AS first_purchase, MAX(market_date) AS last_purchase,
       DATEDIFF(MAX(market_date), MIN(market_date)) AS tenure_days
FROM customer_purchases
GROUP BY customer_id;
```

A144.

```
SELECT customer_id FROM customer_purchases
GROUP BY customer_id
HAVING MAX(YEAR(market_date)) = 2019;          -- customer 13
-- or: HAVING SUM(YEAR(market_date) = 2020) = 0
```

A145.

```
SELECT customer_id, MIN(product_id) AS only_product
FROM customer_purchases
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) = 1;         -- customer 15
```

A146.

```
SELECT customer_id, product_id, market_date, COUNT(*) AS times,
       GROUP_CONCAT(transaction_time ORDER BY transaction_time) AS times_list
FROM customer_purchases
GROUP BY customer_id, product_id, market_date
HAVING COUNT(*) > 1;
```

A147.

```
SELECT booth_number, market_date, COUNT(*) AS vendors
FROM vendor_booth_assignments
GROUP BY booth_number, market_date
HAVING COUNT(*) > 1;

SELECT COUNT(*) FROM (SELECT booth_number, market_date FROM vendor_booth_assignments
                      GROUP BY booth_number, market_date HAVING COUNT(*) > 1) t;     -- 35
```

Every conflict is booth 11, on each market day from 2019-06-01 to 2019-09-28, shared by vendors 4 and 7.

A148.

```
SELECT vendor_id, market_date, GROUP_CONCAT(booth_number ORDER BY booth_number) AS booths
FROM vendor_booth_assignments
GROUP BY vendor_id, market_date
HAVING COUNT(DISTINCT booth_number) > 1;        -- vendor 4: '7,11'
```

A149.

```
SELECT market_year, market_week, GROUP_CONCAT(DISTINCT market_season) AS seasons
FROM market_date_info
GROUP BY market_year, market_week
HAVING COUNT(DISTINCT market_season) > 1;       -- 2019 wk 22 and 2019 wk 44
```

A150.

```
SELECT market_year, market_week, COUNT(*) AS market_days
FROM market_date_info
GROUP BY market_year, market_week
HAVING COUNT(*) <> 2;                           -- 2019 wk 9 and 10 (Saturday only)
```

A151.

```
SELECT DAYNAME(market_date) AS dow, HOUR(transaction_time) AS hr,
       COUNT(*) AS lines, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS revenue
FROM customer_purchases
GROUP BY dow, hr
ORDER BY dow, lines DESC;
```

The busiest Saturday hour is 13:00, with 37 lines.

A152. It fails with **Error 1054: Unknown column 'market_date' in 'having clause'**. HAVING can only refer to grouped columns, aggregates or SELECT aliases.

```
SELECT customer_id, SUM(quantity * cost_to_customer_per_qty) AS spend
FROM customer_purchases
WHERE market_date >= '2020-01-01'
GROUP BY customer_id;
```

WHERE removes rows **before** they're grouped, so less data is aggregated, and it can use an index.

A153. `ONLY_FULL_GROUP_BY` has been on by default since MySQL 5.7.5. `market_date` isn't aggregated and isn't in the GROUP BY, so it's ambiguous. MySQL 5.6 silently returned an arbitrary date.

```
SELECT customer_id, market_date, SUM(quantity) FROM customer_purchases GROUP BY customer_id,
market_date;
SELECT customer_id, MAX(market_date) AS last_date, SUM(quantity) FROM customer_purchases GROUP
BY customer_id;
```

A154.

```
SELECT FLOOR(quantity * cost_to_customer_per_qty / 5) * 5
AS bucket_start,
       CONCAT(FLOOR(quantity * cost_to_customer_per_qty / 5) * 5, '-',
              FLOOR(quantity * cost_to_customer_per_qty / 5) * 5 + 5)
AS bucket,
       COUNT(*) AS lines,
       ROUND(SUM(quantity * cost_to_customer_per_qty), 2)
AS revenue
FROM customer_purchases
GROUP BY bucket_start, bucket
ORDER BY bucket_start;
```

Section 13: ORDER BY, LIMIT, OFFSET

A155.

```
SELECT customer_id, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS spend
FROM customer_purchases
GROUP BY customer_id
ORDER BY spend DESC
LIMIT 5;
```

A156.

```
SELECT * FROM customer
ORDER BY customer_last_name, customer_first_name
LIMIT 10 OFFSET 20;           -- same as LIMIT 20, 10
```

Page 3 has only **6 rows**: Rai, Sipp, Valenzuela, Washington, Wilson and Wise. Add customer_id as a final sort key so the pages stay stable.

A157.

```
SELECT DISTINCT ROUND(quantity * cost_to_customer_per_qty, 2) AS amount
FROM customer_purchases
ORDER BY amount DESC
LIMIT 1 OFFSET 1;             -- 39.63
```

A158. `LIMIT 1, 1` and `LIMIT 1 OFFSET 1` both return the **2nd** row. `LIMIT 1, 3` returns rows **2 to 4**. In the comma form, the first number is the offset.

A159.

```
SELECT * FROM market_date_info
ORDER BY STR_TO_DATE(market_start_time, '%l:%i %p'), market_date;
```

A160.

```
SELECT * FROM customer_purchases
WHERE quantity >= (SELECT quantity FROM customer_purchases ORDER BY quantity DESC LIMIT 1
OFFSET 2)
ORDER BY quantity DESC;
```

Plain `LIMIT 3` cuts off rows that tie with the 3rd value, and **which** of the tied rows survives is arbitrary.

A161. `GROUP BY 1` and `ORDER BY 2` refer to the 1st and 2nd columns in the SELECT list. If someone later adds or reorders columns, the query silently groups or sorts by the wrong thing. It's fine for ad-hoc work, but avoid it in production code.

A162. `market_week` is VARCHAR, so it sorts as text, and '9' > '52'.

```
SELECT DISTINCT market_week FROM market_date_info ORDER BY market_week + 0;
-- or: ORDER BY CAST(market_week AS UNSIGNED)
```

Section 14: Order of Execution

A163. The logical order is **FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET**.

```
SELECT customer_id, SUM(quantity * cost_to_customer_per_qty) AS spend    -- 5  compute output
FROM customer_purchases                                                  -- 1  read table
WHERE market_date >= '2020-01-01'                                        -- 2  filter rows
GROUP BY customer_id                                                     -- 3  make groups
HAVING SUM(quantity * cost_to_customer_per_qty) > 100                    -- 4  filter groups
ORDER BY spend DESC                                                      -- 6  sort
LIMIT 10;                                                                -- 7  trim
```

A164. It fails with **Error 1111: Invalid use of group function**, because aggregates don't exist yet when WHERE runs. Use HAVING:

```
SELECT customer_id, SUM(quantity) FROM customer_purchases GROUP BY customer_id HAVING
SUM(quantity) > 10;
```

A165. It runs in MySQL, which allows SELECT aliases in HAVING as an extension. PostgreSQL and SQL Server reject it. The portable version repeats the expression in HAVING.

A166. It works in MySQL, and PostgreSQL also allows it. SQL Server and Oracle before 23c don't. For portability, write `GROUP BY YEAR(market_date)`.

A167. ORDER BY runs **after** SELECT, so the aliases already exist by then. WHERE runs **before** SELECT, when they don't.

A168.

1. FROM reads all rows.
2. WHERE keeps the 98 rows for product 2.
3. GROUP BY makes one group per customer.
4. HAVING keeps 9 customers: six with n = 6 (customers 2, 3, 4, 5, 14, 26) and three with n = 5.
5. ORDER BY n DESC.
6. LIMIT 2 keeps two rows.

The result is **not deterministic**. It returns 2 of the 6 customers tied at 6, and which 2 can change between runs. Add a tiebreaker: `ORDER BY n DESC, customer_id`.

A169. AVG collapses every row into **one** row, and ORDER BY and LIMIT then apply to that single row. So the query returns the overall average, 14.21. Sort and limit inside a derived table first:

```
SELECT AVG(amount) AS avg_top10
FROM (SELECT quantity * cost_to_customer_per_qty AS amount
      FROM customer_purchases
      ORDER BY amount DESC
      LIMIT 10) t;
```

A170. The first query returns **one row containing 0**. The second returns **no rows at all**, because there are no groups to report.

Section 15: Subqueries

A171.

```
SELECT * FROM customer_purchases
WHERE quantity * cost_to_customer_per_qty >
      (SELECT AVG(quantity * cost_to_customer_per_qty) FROM customer_purchases);    -- 125 rows
```

A172.

```
SELECT * FROM customer
WHERE customer_id NOT IN (SELECT customer_id FROM customer_purchases);

SELECT * FROM customer c
WHERE NOT EXISTS (SELECT 1 FROM customer_purchases cp WHERE cp.customer_id = c.customer_id);
```

Both return an empty result, because all 26 customers have bought something. NOT EXISTS is the safer habit, since NOT IN breaks if the subquery ever returns a NULL.

A173.

```
SELECT customer_first_name, customer_last_name FROM customer
WHERE customer_id IN (SELECT customer_id FROM customer_purchases
                      GROUP BY customer_id
                      HAVING SUM(quantity * cost_to_customer_per_qty) > 200);
-- Jane Connor, Manuel Diaz, Deanna Washington, Abigail Harris
```

A174.

```
SELECT product_name FROM product
WHERE product_id IN (SELECT DISTINCT product_id FROM customer_purchases);
```

A175.

```
SELECT c1.customer_id, c1.market_date,
       ROUND(c1.quantity * c1.cost_to_customer_per_qty, 2) AS line_value,
       ROUND((SELECT AVG(c2.quantity * c2.cost_to_customer_per_qty)
              FROM customer_purchases c2 WHERE c2.customer_id = c1.customer_id), 2) AS
cust_avg,
       ROUND(c1.quantity * c1.cost_to_customer_per_qty
             - (SELECT AVG(c2.quantity * c2.cost_to_customer_per_qty)
                FROM customer_purchases c2 WHERE c2.customer_id = c1.customer_id), 2) AS
diff_from_avg
FROM customer_purchases c1;
```

A176.

```
SELECT * FROM customer_purchases cp
WHERE cp.cost_to_customer_per_qty <
      (SELECT vi.original_price FROM vendor_inventory vi
       WHERE vi.market_date = cp.market_date AND vi.vendor_id = cp.vendor_id AND vi.product_id
= cp.product_id);
-- the two 3.00 sales on 2019-07-13
```

A177.

```
SELECT d.product_id, d.market_date, d.revenue
FROM (SELECT product_id, market_date, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS
revenue
      FROM customer_purchases GROUP BY product_id, market_date) d
WHERE d.revenue = (SELECT MAX(x.revenue)
                   FROM (SELECT product_id, ROUND(SUM(quantity * cost_to_customer_per_qty), 2)
AS revenue
                         FROM customer_purchases GROUP BY product_id, market_date) x
                   WHERE x.product_id = d.product_id);
```

Another option: `GROUP BY product_id, market_date HAVING SUM(...) >= ALL (SELECT SUM(...) FROM customer_purchases c2 WHERE c2.product_id = c.product_id GROUP BY c2.market_date)`.

A178.

```
SELECT customer_id, SUM(quantity * cost_to_customer_per_qty) AS spend
FROM customer_purchases
GROUP BY customer_id
HAVING SUM(quantity * cost_to_customer_per_qty) =
       (SELECT MAX(s) FROM (SELECT SUM(quantity * cost_to_customer_per_qty) AS s
                            FROM customer_purchases GROUP BY customer_id) t);     -- customer 4
```

A179.

```
SELECT market_date, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS revenue
FROM customer_purchases
GROUP BY market_date
HAVING SUM(quantity * cost_to_customer_per_qty) >
       (SELECT AVG(r) FROM (SELECT SUM(quantity * cost_to_customer_per_qty) AS r
                            FROM customer_purchases GROUP BY market_date) t);
```

A180.

```
SELECT market_date, sold, brought, ROUND(100 * sold / brought, 1) AS sell_through_pct
FROM (
  SELECT cp.market_date,
         SUM(cp.quantity) AS sold,
         (SELECT vi.quantity FROM vendor_inventory vi
          WHERE vi.product_id = 1 AND vi.vendor_id = 7 AND vi.market_date = cp.market_date) AS
brought
  FROM customer_purchases cp
  WHERE cp.product_id = 1
  GROUP BY cp.market_date
) t
ORDER BY market_date;
```

A181.

```
SELECT * FROM vendor v
WHERE EXISTS (SELECT 1 FROM vendor_inventory vi
              WHERE vi.vendor_id = v.vendor_id AND vi.product_id = 4
                AND vi.market_date >= '2020-01-01' AND vi.market_date < '2021-01-01');   --
vendor 7
```

A182.

```
SELECT customer_id,
       ROUND(100 * SUM(quantity * cost_to_customer_per_qty)
             / (SELECT SUM(quantity * cost_to_customer_per_qty) FROM customer_purchases), 2) AS
pct_of_revenue
FROM customer_purchases
GROUP BY customer_id
ORDER BY pct_of_revenue DESC;
```

A183.

```
SELECT market_date, COUNT(DISTINCT customer_id) AS customers
FROM customer_purchases
GROUP BY market_date
HAVING COUNT(DISTINCT customer_id) =
       (SELECT MAX(c) FROM (SELECT COUNT(DISTINCT customer_id) AS c
                            FROM customer_purchases GROUP BY market_date) t);   -- 2019-07-03
(9)
```

A184.

```
SELECT customer_id, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS spend
FROM customer_purchases
GROUP BY customer_id
HAVING SUM(quantity * cost_to_customer_per_qty) =
  (SELECT MAX(s) FROM (SELECT SUM(quantity * cost_to_customer_per_qty) AS s FROM
customer_purchases GROUP BY customer_id) a
   WHERE s < (SELECT MAX(s2) FROM (SELECT SUM(quantity * cost_to_customer_per_qty) AS s2
                                   FROM customer_purchases GROUP BY customer_id) b));   --
customer 2, 305.03
```

Section 16: Statistics

A185.

```
SELECT ROUND(AVG(quantity), 4)          AS mean_qty,     -- 2.5999
       ROUND(STDDEV_POP(quantity), 4)   AS sd_pop,       -- 1.5682
       ROUND(STDDEV_SAMP(quantity), 4)  AS sd_sample,    -- 1.5711
       ROUND(VAR_POP(quantity), 4)      AS var_pop,
       ROUND(VAR_SAMP(quantity), 4)     AS var_sample
FROM customer_purchases;
```

In MySQL, `STD()` and `STDDEV()` are the same as `STDDEV_POP()`, which divides by n. `STDDEV_SAMP()` divides by n − 1, which is the right choice when the data is a sample of a larger population.

A186. A value is the median if at least half the rows are ≤ it **and** at least half are ≥ it:

```
SELECT AVG(DISTINCT q.quantity) AS median_qty          -- 2.44
FROM customer_purchases q
WHERE (SELECT COUNT(*) FROM customer_purchases x WHERE x.quantity <= q.quantity)
          >= (SELECT COUNT(*) FROM customer_purchases) / 2
  AND (SELECT COUNT(*) FROM customer_purchases x WHERE x.quantity >= q.quantity)
          >= (SELECT COUNT(*) FROM customer_purchases) / 2;
```

When the count is even, the condition selects the two middle values, and `AVG(DISTINCT …)` averages them. `DISTINCT` stops tied duplicates from skewing that average.

A187.

```
SELECT quantity, COUNT(*) AS freq
FROM customer_purchases
GROUP BY quantity
HAVING COUNT(*) = (SELECT MAX(c) FROM (SELECT COUNT(*) AS c FROM customer_purchases GROUP BY
quantity) t);
-- 2.28 (5 times)
```

`ORDER BY COUNT(*) DESC LIMIT 1` would silently drop any tied modes.

A188.

```
SELECT customer_id, market_date,
       ROUND(quantity * cost_to_customer_per_qty, 2) AS amount,
       ROUND((quantity * cost_to_customer_per_qty
              - (SELECT AVG(quantity * cost_to_customer_per_qty) FROM customer_purchases))
             / (SELECT STDDEV_POP(quantity * cost_to_customer_per_qty) FROM
customer_purchases), 2) AS z_score
FROM customer_purchases
WHERE (quantity * cost_to_customer_per_qty
       - (SELECT AVG(quantity * cost_to_customer_per_qty) FROM customer_purchases))
      / (SELECT STDDEV_POP(quantity * cost_to_customer_per_qty) FROM customer_purchases) > 2
ORDER BY z_score DESC;          -- 11 rows (mean 14.21, sd 9.12)
```

A189.

```
SELECT product_id, COUNT(*) AS days,
       ROUND(AVG(quantity), 2) AS mean_qty,
       ROUND(STDDEV_POP(quantity) / AVG(quantity), 4) AS cv
FROM vendor_inventory
GROUP BY product_id
ORDER BY cv DESC;
```

Product 1 (Habanero) has the highest CV, 0.18, so its supply is the least predictable. Product 7 has only 4 days of data, so its CV isn't reliable.

A190.

```
SELECT market_season,
       COUNT(market_max_temp)                              AS recorded_days,
       ROUND(AVG(market_max_temp), 1)                      AS mean_max,
       MIN(market_max_temp + 0)                            AS min_max,
       MAX(market_max_temp + 0)                            AS max_max,
       MAX(market_max_temp + 0) - MIN(market_max_temp + 0)  AS range_max,
       (MAX(market_max_temp + 0) + MIN(market_max_temp + 0)) / 2 AS mid_range
FROM market_date_info
GROUP BY market_season;
```

The `+ 0` makes MIN and MAX compare the values as numbers. 'Late Fall/Holiday' has no readings, so its statistics are all NULL.

A191.

```
SELECT ROUND((AVG(market_min_temp * market_max_temp) - AVG(market_min_temp) *
AVG(market_max_temp))
             / (STDDEV_POP(market_min_temp) * STDDEV_POP(market_max_temp)), 4) AS pearson_r
-- 0.9573
FROM market_date_info
WHERE market_min_temp IS NOT NULL AND market_max_temp IS NOT NULL;
```

The WHERE clause makes all four aggregates use the **same** rows.

A192.

```
SELECT c1.*, ROUND(c1.quantity * c1.cost_to_customer_per_qty, 2) AS amount
FROM customer_purchases c1
WHERE (SELECT COUNT(*) FROM customer_purchases c2
       WHERE c2.quantity * c2.cost_to_customer_per_qty > c1.quantity *
c1.cost_to_customer_per_qty)
      < 0.10 * (SELECT COUNT(*) FROM customer_purchases)
ORDER BY amount DESC;            -- 28 rows
```

A193.

```
SELECT d.market_date, d.revenue,
       (SELECT ROUND(SUM(x.quantity * x.cost_to_customer_per_qty), 2)
        FROM customer_purchases x
        WHERE x.market_date = (SELECT MAX(y.market_date) FROM customer_purchases y
                               WHERE y.market_date < d.market_date))            AS
prev_revenue,
       ROUND(100 * (d.revenue - (SELECT SUM(x.quantity * x.cost_to_customer_per_qty)
                                 FROM customer_purchases x
                                 WHERE x.market_date = (SELECT MAX(y.market_date) FROM
customer_purchases y
                                                        WHERE y.market_date < d.market_date)))
             / (SELECT SUM(x.quantity * x.cost_to_customer_per_qty)
                FROM customer_purchases x
                WHERE x.market_date = (SELECT MAX(y.market_date) FROM customer_purchases y
                                       WHERE y.market_date < d.market_date)), 1) AS pct_change
FROM (SELECT market_date, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS revenue
      FROM customer_purchases GROUP BY market_date) d
ORDER BY d.market_date;
```

The first date has no previous day, so its previous revenue and % change are both NULL.

A194.

```
SELECT d.market_date, d.revenue,
       (SELECT ROUND(SUM(x.quantity * x.cost_to_customer_per_qty), 2)
        FROM customer_purchases x WHERE x.market_date <= d.market_date) AS running_total
FROM (SELECT market_date, ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS revenue
      FROM customer_purchases GROUP BY market_date) d
ORDER BY d.market_date;
```

Section 17: Interview Logic Scenarios

A195.

```
SELECT customer_id, COUNT(*) AS weeks_with_both_days
FROM (SELECT customer_id, YEARWEEK(market_date, 3) AS yw
      FROM customer_purchases
      GROUP BY customer_id, YEARWEEK(market_date, 3)
      HAVING COUNT(DISTINCT DAYOFWEEK(market_date)) = 2) t
GROUP BY customer_id
ORDER BY weeks_with_both_days DESC;           -- 11 customers
```

ISO weeks run Monday to Sunday, so each week contains exactly one Wednesday and one Saturday.

A196.

```
SELECT ROUND(100 * SUM(visits > 1) / COUNT(*), 2)     AS repeat_pct,       -- 100.00
       ROUND(100 * SUM(years = 2)  / COUNT(*), 2)     AS both_years_pct    -- 96.15 (25 of 26)
FROM (SELECT customer_id,
             COUNT(DISTINCT market_date)       AS visits,
             COUNT(DISTINCT YEAR(market_date)) AS years
      FROM customer_purchases GROUP BY customer_id) t;
```

A197.

```
SELECT customer_id,
       ROUND(SUM(CASE WHEN YEAR(market_date) = 2019 THEN quantity * cost_to_customer_per_qty
ELSE 0 END), 2) AS spend_2019,
       ROUND(SUM(CASE WHEN YEAR(market_date) = 2020 THEN quantity * cost_to_customer_per_qty
ELSE 0 END), 2) AS spend_2020,
       ROUND(100 * (SUM(CASE WHEN YEAR(market_date) = 2020 THEN quantity *
cost_to_customer_per_qty ELSE 0 END)
                  - SUM(CASE WHEN YEAR(market_date) = 2019 THEN quantity *
cost_to_customer_per_qty ELSE 0 END))
             / NULLIF(SUM(CASE WHEN YEAR(market_date) = 2019 THEN quantity *
cost_to_customer_per_qty ELSE 0 END), 0), 1) AS growth_pct
FROM customer_purchases
GROUP BY customer_id
HAVING spend_2020 > spend_2019
ORDER BY growth_pct DESC;        -- customers 1, 4, 7, 9, 11, 14, 15, 21, 26
```

A198.

```
SELECT vi.market_date, vi.quantity AS brought
FROM vendor_inventory vi
WHERE vi.vendor_id = 7 AND vi.product_id = 1
  AND NOT EXISTS (SELECT 1 FROM customer_purchases cp
                  WHERE cp.vendor_id = 7 AND cp.product_id = 1 AND cp.market_date =
vi.market_date);
-- 2019-08-17, 2020-07-29, 2020-08-19
```

A199.

```
SELECT MONTH(market_date) AS mon, MONTHNAME(MIN(market_date)) AS month_name,
       ROUND(SUM(CASE WHEN YEAR(market_date) = 2019 THEN quantity * cost_to_customer_per_qty
ELSE 0 END), 2) AS rev_2019,
       ROUND(SUM(CASE WHEN YEAR(market_date) = 2020 THEN quantity * cost_to_customer_per_qty
ELSE 0 END), 2) AS rev_2020,
       ROUND(100 * (SUM(CASE WHEN YEAR(market_date) = 2020 THEN quantity *
cost_to_customer_per_qty ELSE 0 END)
                  / NULLIF(SUM(CASE WHEN YEAR(market_date) = 2019 THEN quantity *
cost_to_customer_per_qty ELSE 0 END), 0) - 1), 1) AS yoy_pct
FROM customer_purchases
WHERE MONTH(market_date) IN (7, 8, 9)
GROUP BY MONTH(market_date)
ORDER BY mon;
-- Jul 976.80 → 523.06, Aug 925.31 → 450.23, Sep 383.56 → 607.22
```

The September 2019 figure is low only because the extract is cut off, so check your data before you trust a trend.

A200.

```
SELECT
  (SELECT ROUND(SUM(quantity * cost_to_customer_per_qty), 2) FROM customer_purchases)
AS total_revenue,
  (SELECT COUNT(DISTINCT customer_id) FROM customer_purchases)
AS total_customers,
  (SELECT customer_id FROM customer_purchases GROUP BY customer_id
   ORDER BY SUM(quantity * cost_to_customer_per_qty) DESC LIMIT 1)
AS best_customer,
  (SELECT market_date FROM customer_purchases GROUP BY market_date
   ORDER BY SUM(quantity * cost_to_customer_per_qty) DESC LIMIT 1)
AS best_market_date,
  (SELECT HOUR(transaction_time) FROM customer_purchases GROUP BY HOUR(transaction_time)
   ORDER BY COUNT(*) DESC LIMIT 1)
AS busiest_hour;
-- 3866.18 | 26 | 4 | 2019-08-10 | 18
```

Each scalar subquery must return exactly one row and one column, which is why `LIMIT 1` is required.